

Agent Fabric

Session 2 — Integration

MCP

FastMCP

langchain-mcp-adapters

SQL Agent

Capstone

Nguyễn Hữu Nhật Thành · Nguyễn Ngọc Quý

11 June 2026 · 14:00 – 16:00 ICT · via Zoom

What we build today

01

MCP — Concept

· Module 4

What MCP is, why it exists, the Host-Client-Server architecture, and the three primitives: Tools, Resources, Prompts.

02

FastMCP + Adapters

· Module 4

Build an MCP server with FastMCP. Consume it inside a LangGraph agent via langchain-mcp-adapters. (The graph holds state; the MCP client session itself is per-call.)

03

SQL Agent — Concept

· Module 5

Natural language → SQL: schema injection, SQL generation, validation, and feeding results back to the LLM.

04

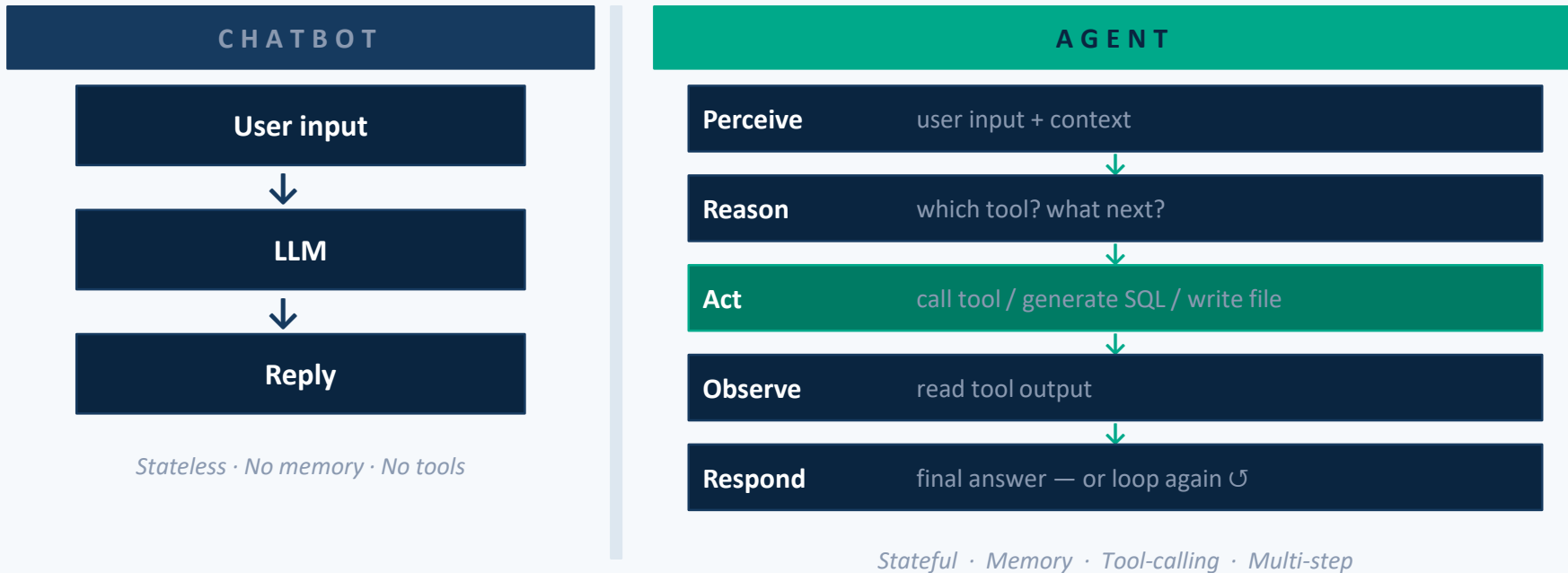
LangGraph SQL Agent

· Module 5

SQLDatabaseToolkit (langchain-community) wired into create_agent (langchain.agents). Four pre-built tools, zero manual SQL pipeline.

Agent vs Chatbot

A chatbot replies once and stops. An agent acts, observes the result, and decides whether to loop — until the task is done.



Model Context Protocol

USB-C for AI agents.

Every LLM app needs to connect to tools and data sources — each with its own API, auth, and schema. MCP defines a single open protocol for all of these connections. Any MCP-compliant host can talk to any MCP server without bespoke glue code per integration.

Tools

Actions the model can invoke: run a query, write a file, send a message. The model decides when to call them.

Resources

Read-only data the model can fetch: document contents, DB schemas, file trees. Grounding context, not actions.

Prompts

Reusable prompt templates exposed by the server. The agent retrieves them by name and injects into context.

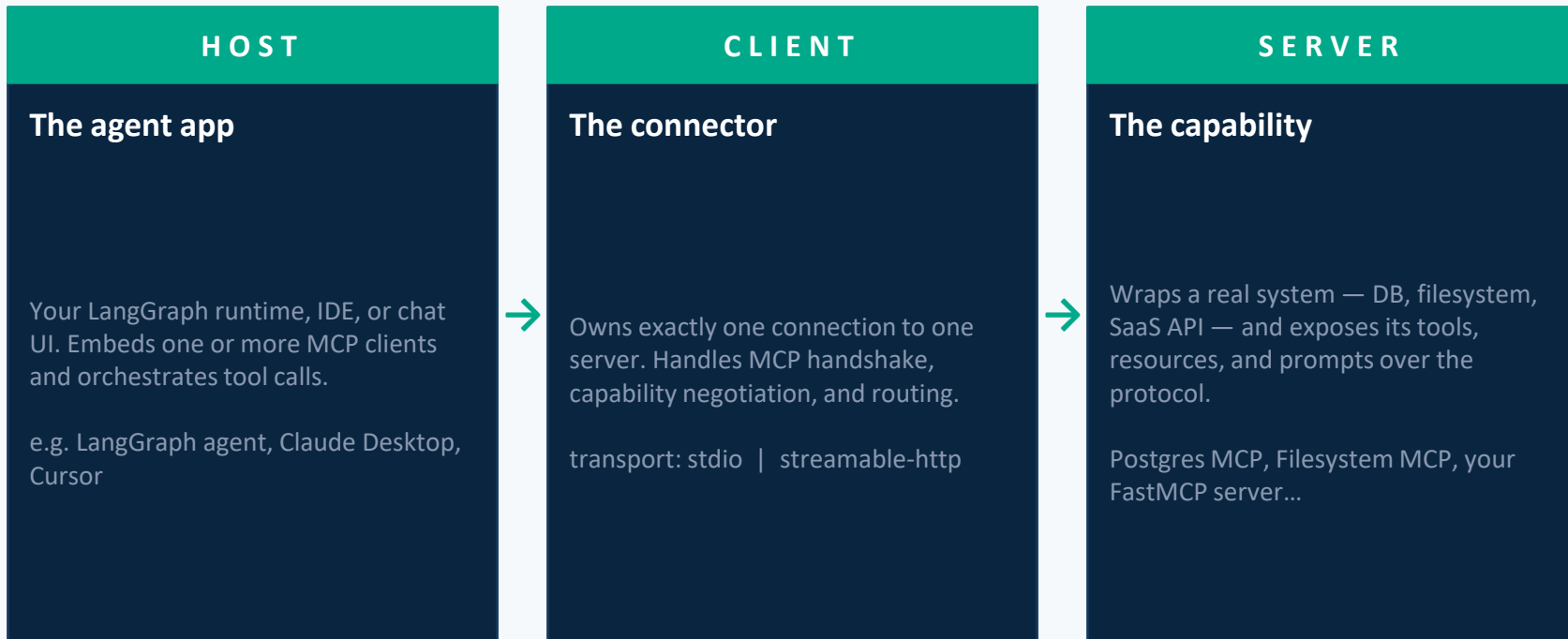
MCP vs Tool Calling

The question everyone will ask. Tool Calling is a capability. MCP is a distribution protocol for that capability.

Tool Calling		MCP Protocol
Definition	Local function exposed to an LLM	Standard open protocol for connecting hosts to servers
Scope	App-specific — one integration per app	Reusable — one server talks to any MCP-compliant host
Schema	Defined manually per integration	Auto-discovered at connection time
Coupling	Tight — server logic baked into app	Loose — swap server URL = zero code change in agent
Distribution	Cannot be shared across apps	One server → Claude Desktop, Cursor, LangGraph, all at once

Use MCP when: the same tool must serve multiple agent hosts · schema auto-discovery saves integration time · the server should deploy independently of the agent. One app only? Plain tool call is fine.

Host, client, server



One client ↔ one server. One host can embed many clients — connect multiple servers simultaneously.

MCP is becoming the standard

Every major AI tool is adding MCP support. One server you write today connects to all of these.

Claude Desktop

Anthropic ·

First mover

Built-in MCP client. Connect any MCP server to Claude Desktop in seconds via config.

Cursor

AnySphere ·

IDE agent

MCP tools appear as context in the editor. Cursor agent calls them for code tasks.

Claude Code

Anthropic ·

CLI agent

Command-line coding agent. Reads your MCP config and calls servers during tasks.

LangGraph

LangChain ·

Orchestrator

MultiServerMCPClient wraps any server into LangChain tools for use in agents and graphs.

OpenAI Agents SDK

OpenAI ·

Python SDK

Native MCP client in the Agents SDK. Same pattern: connect, list tools, call tools.

FastMCP

jlowin ·

Server framework

The reference Python library for building MCP servers. Decorators, zero boilerplate.

Three things a server can expose

Any MCP server declares some combination of these. Clients discover them at connection time.



Tools

Actions the model invokes (function call pattern)

```
run_query(sql) → rows
```

```
write_file(path, content)
```

```
send_slack_message(channel, text)
```

The model decides when and how often to call.
Each call returns an observation.



Resources

Read-only data the model fetches by URI

```
schema://warehouse → DDL
```

```
file://docs/README.md → content
```

```
config://settings → JSON
```

Cheaper than a tool call — no model decision needed. Good for grounding context.



Prompts

Reusable prompt templates the server defines

```
sql_system_prompt(dialect, top_k)
```

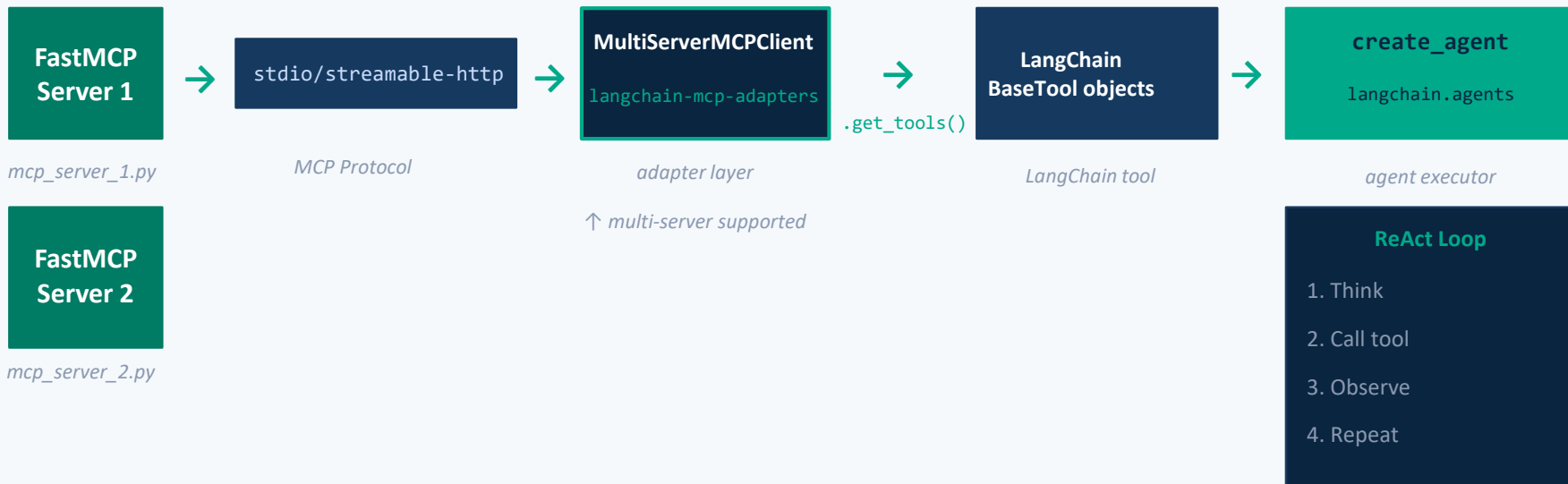
```
summarize_rows(question, rows)
```

```
code_review(diff)
```

The agent pulls them by name. Centralises prompt engineering in the server.

How MCP connects at runtime

langchain-mcp-adapters bridges MCP protocol → native LangChain tools. create_agent runs the ReAct loop.



Why FastMCP over the raw SDK?

The raw MCP Python SDK works. FastMCP removes the boilerplate so you ship a server in minutes, not hours.

Raw MCP SDK

```
from mcp.server import Server
from mcp.types import Tool, TextContent

app = Server("analytics")

# Must manually declare tool schemas
@app.list_tools()
async def list_tools() -> list[Tool]:
    return [Tool(name="run_query",
        description="Run SQL",
        inputSchema={...})] # manual

# And manually handle call dispatch
@app.call_tool()
async def call_tool(name, args):
    if name == "run_query": ...
```

FastMCP ✓

```
from fastmcp import FastMCP

mcp = FastMCP("analytics")

# Schema auto-generated from type hints
@mcp.tool()
def run_query(sql: str) -> list:
    """Run a SELECT query."""
    return execute(sql)

# That's the whole server.
mcp.run()
```

Writing a FastMCP server

mcp_server.py

```
from fastmcp import FastMCP
from sqlalchemy import create_engine, text

mcp = FastMCP("analytics-server")
engine = create_engine(DATABASE_URL)

@mcp.tool()
def run_query(sql: str) -> list[dict]:
    """Execute a SELECT – returns rows."""
    if not sql.upper().startswith("SELECT"):
        raise ValueError("SELECT only")
    with engine.connect() as conn:
        rows = conn.execute(text(sql))
        return [dict(r) for r in rows]

@mcp.resource("schema://warehouse")
def get_schema() -> str:
    return fetch_ddl(engine) # DDL for all tables

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Key patterns

@mcp.tool()

Registers the function. Docstring = LLM-visible description.
Type hints = JSON Schema. No boilerplate.

Guard first

Reject non-SELECT before touching the DB. Never trust model output directly at the DB layer.

@mcp.resource()

Exposes warehouse DDL as a read-only resource. Agent fetches by URI to orient before generating SQL.

mcp.run(transport=)

stdio for local/CLI use. Change to streamable-http for a remote/network service.

langchain-mcp-adapters client

```
mcp_client.py
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

# connect to one or more MCP servers
client = MultiServerMCPClient({
    "analytics": {
        "command": "python",
        "args": ["mcp_server.py"],
        "transport": "stdio",
    }
})

# MCP tools → native LangChain BaseTool objects
tools = await client.get_tools()

# create_agent replaces deprecated
# create_react_agent from langgraph.prebuilt (v1.0)
agent = create_agent(
    model="anthropic:claude-sonnet-4-5",
    tools=tools,
    system_prompt="Analytics assistant.")

# stream: agent.astream(msgs, stream_mode='updates')
```

Four things to know

MultiServerMCPClient

Takes a dict of server configs. Each entry can use stdio or streamable-http transport. Connects all on first `get_tools()` call.

get_tools()

Fetches server's tool list. Returns native LangChain BaseTool objects — pass directly to `create_agent`. No schema config needed.

create_agent (new)

From `langchain.agents` — replaces deprecated `create_react_agent`. Same ReAct loop, cleaner API, middleware support.

astream(stream_mode=)

"updates" yields each node's delta. "values" yields full state per step. Use "updates" for streaming UIs.

MODULE 5

SQL Agent

Natural language → SQL → Grounded answer

Why SQL Agent?

"Which 5 customers generated the most revenue last quarter?"

Without SQL Agent

1. Open SQL editor
2. Recall correct table & column names
3. Write and debug the JOIN + GROUP BY
4. Copy result to spreadsheet
5. Format and email it

Needs SQL knowledge. Friction every time. Most PMs can't do step 2–3 at all.

With SQL Agent ✓

1. Ask in plain English
2. Agent lists tables, reads schema
3. Generates + validates SQL
4. Executes safely, returns answer

No SQL knowledge needed. Guardrails built in. Runs in seconds. Anyone on the team can use it.

SQLDatabaseToolkit makes this possible with 4 pre-built tools. Next slide ↓

SQLDatabaseToolkit — 4 tools

"What were the top 5 customers by revenue last quarter?"

01

`sql_db_list_tables`

ListSQLDatabaseTool

Lists all tables in the connected DB. Agent calls this first to discover what exists before writing any SQL.

02

`sql_db_schema`

InfoSQLDatabaseTool

Returns CREATE TABLE DDL + sample rows for selected tables. Gives the LLM your actual schema, not guesses.

03

`sql_db_query_checker`

QuerySQLCheckerTool

Uses the LLM to review the generated SQL for errors before execution. Catches wrong columns, syntax bugs, dialect issues.

04

`sql_db_query`

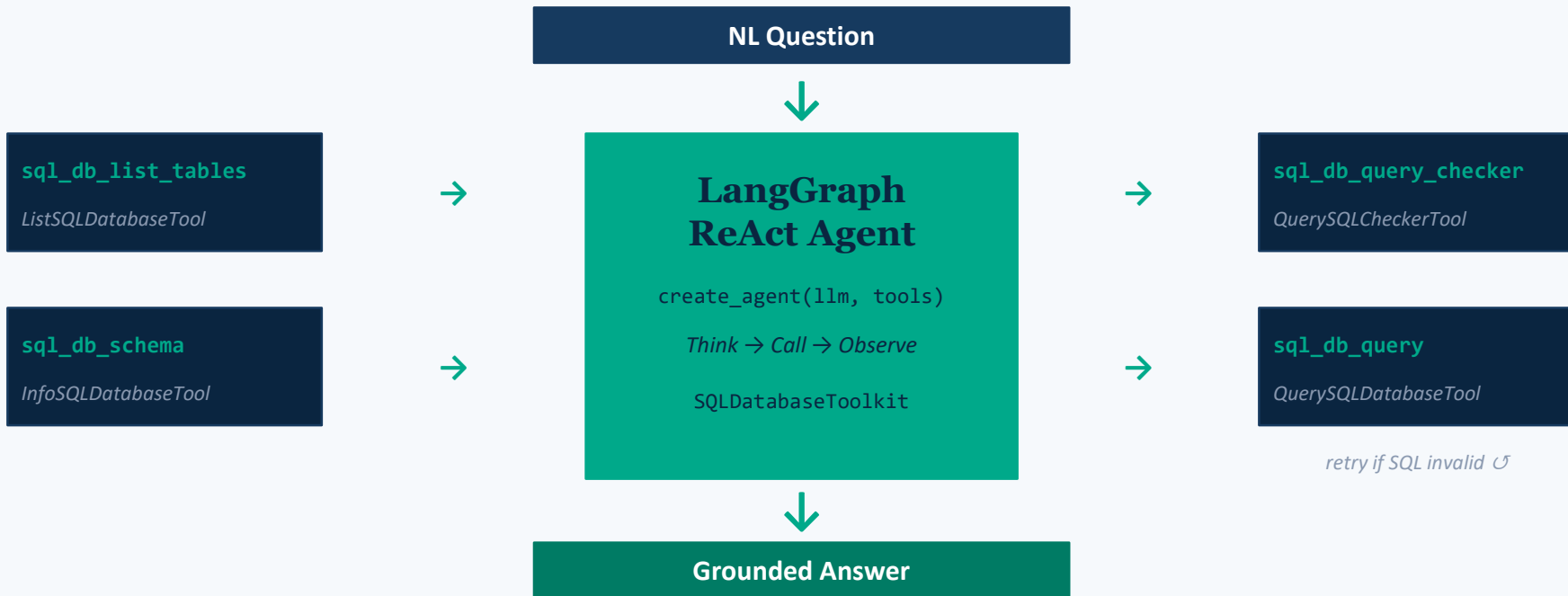
QuerySQLDatabaseTool

Executes the validated SQL and returns rows. Agent feeds results back into context to compose the final answer.

These are the actual tool names the agent calls — not class names. The agent uses them autonomously in the ReAct loop.

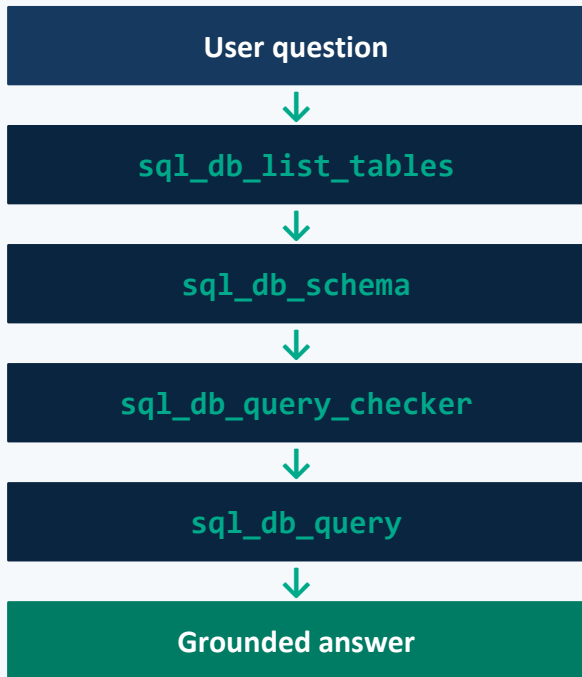
The autonomous ReAct loop

The agent decides autonomously when to list tables, inspect schema, check SQL, and execute — no manual pipeline.



Standard SQL Agent workflow

The agent typically follows this sequence. Each step feeds the next — schema first, generate, check, then execute.



User: "Top 5 customers by revenue?"

1. `sql_db_list_tables`
→ [customers, orders, products, ...]
2. `sql_db_schema`
→ CREATE TABLE customers (id, name, ...)
CREATE TABLE orders (id, customer_id, amount, ...)
3. `sql_db_query_checker`
→ SELECT c.name, SUM(o.amount) AS revenue
FROM customers c JOIN orders o ON c.id = o.customer_id
GROUP BY c.name ORDER BY revenue DESC LIMIT 5 ✓
4. `sql_db_query`
→ 5 rows returned
5. `summarize`
→ "The top 5 customers by revenue are..."

Enterprise SQL Agent: 7 – 10 tools

In production, don't call QuerySQLDatabaseTool directly. Wrap it in a SafeExecuteSQLTool that enforces all guards.

FOUNDATION · SQLDatabaseToolkit

`sql_db_list_tables``sql_db_schema``sql_db_query_checker``sql_db_query ← wrap this!`

→
wrap &
extend

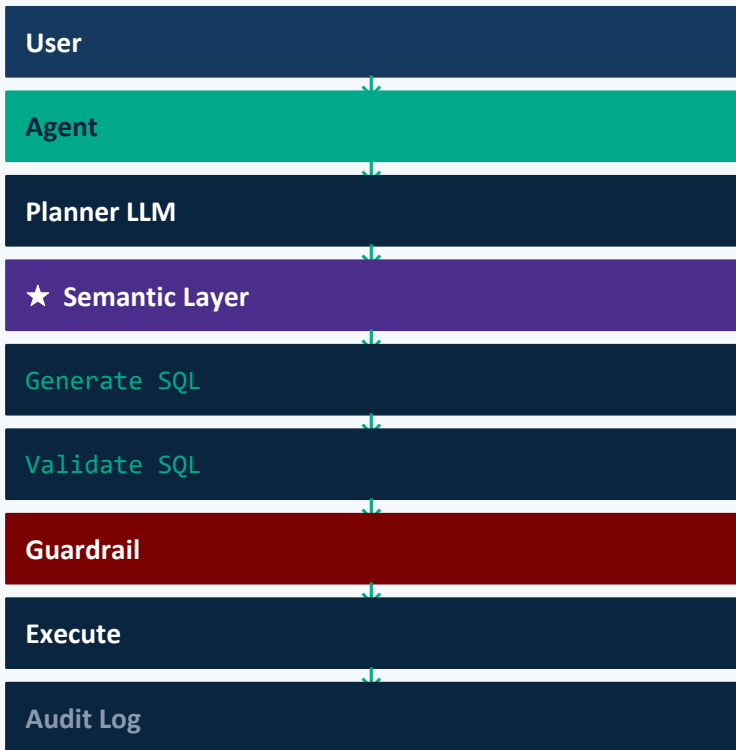
ENTERPRISE · 7 – 10 tools

`get_schema_tool` wraps sql_db_list_tables + sql_db_schema`generate_sql_tool` LLM generates dialect-aware SQL`validate_sql_tool` wraps sql_db_query_checker + sqlglot AST`SafeExecuteSQLTool` wraps sql_db_query ← the key guard layer`tenant_guard_tool` injects tenant_id, rejects cross-tenant SQL`rbac_tool` checks user role before execute`audit_log_tool` logs SQL, user_id, row count, timestamp

SafeExecuteSQLTool automatically: inject tenant_id · block UPDATE/DELETE/DROP · enforce LIMIT · check RBAC/ABAC · log every query

Full enterprise SQL Agent flow

In enterprise BI, the LLM never touches raw tables. A Semantic Layer exposes business metrics instead.



★ Semantic Layer — why it matters

Instead of giving the LLM raw table names, expose business-friendly metrics. The LLM generates SQL against these — not raw columns. This is how enterprise BI tools (dbt metrics, Cube, Lightdash) work.

sales_revenue

```
SUM(orders.amount) WHERE  
status='completed'
```

customer_lifetime_value

```
SUM(payments.amount) GROUP BY  
customer_id
```

monthly_recurring_revenue

```
SUM(subscriptions.mrr) WHERE  
active=true
```

churn_rate

```
cancelled / total_customers (last  
30 days)
```

Agent generates SQL against metric names, not raw table columns. Safer, more stable, easier to audit.

SQLDatabaseToolkit + create_agent

sql_agent.py

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain_community.utilities import SQLDatabase
from langchain.agents import create_agent
from langchain import hub

# 1. connect
db = SQLDatabase.from_uri("postgresql+psycopg2://...")

# 2. toolkit → 4 tools auto-wired
toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

# 3. system prompt from LangChain Hub
prompt = hub.pull("langchain-ai/sql-agent-system-prompt")
sys_msg = prompt.format(dialect=toolkit.dialect, top_k=5)

# 4. create_agent (langchain.agents — v1.0 replacement)
agent = create_agent(
    model="openai:gpt-4o",
    tools=tools,
    system_prompt=sys_msg)
```

Four moving parts

SQLDatabase.from_uri()

Wraps SQLAlchemy. Supports Postgres, MySQL, SQLite, BigQuery. Handles dialect introspection.

toolkit.get_tools()

Returns ListSQL, InfoSQL, CheckSQL, QuerySQL — four tools. Zero manual tool code.

hub.pull(sql-agent-prompt)

Official prompt sets dialect + top_k in the system message. Replaces manual SQL_PROMPT engineering.

create_agent — new API

From langchain.agents (v1.0). Same ReAct loop as deprecated create_react_agent from langgraph.prebuilt.

Safety rails you cannot skip

The toolkit handles schema inspection and SQL checking — but DB-level and infra guards remain your responsibility.

01

Read-only DB user

CREATE ROLE agent_ro
LOGIN; GRANT SELECT ON ALL
TABLES. The agent must never
INSERT, UPDATE, DELETE, or
DROP — enforce at DB layer.

02

QuerySQLCheckerTool

Built into the toolkit — asks
the LLM to review generated
SQL before execution. Catches
wrong columns, syntax errors,
dialect mismatches. Add
sqlglot AST parse on top for
deterministic enforcement.

03

Hard timeout + top_k

Set statement_timeout in
Postgres (e.g. '5s'). The
toolkit's top_k parameter caps
returned rows. A runaway
query must never block the
connection pool.

04

Audit every query

Log generated SQL, user_id,
timestamp, and row count to
an audit table before
returning results. Required for
compliance and LangSmith
eval debugging.

How the agent routes between tools

The agent's LLM reads tool descriptions and selects the right one per question. No routing code needed.

Agent (LLM selects tool based on question)

"How many orders were placed yesterday?"

vs

"Send the revenue report to #data-team on Slack"

Agent reads tool descriptions

Agent reads tool descriptions

sql_db_list_tables → matches 'orders'

Slack MCP tool → matches 'send message'

sql_db_schema(orders) → sees date column

Fetches channel list → finds #data-team

sql_db_query_checker → validates

Formats report summary as message

sql_db_query → COUNT(*) WHERE date = today-1

mcp_slack.send_message(channel, text)

"Yesterday: 3,847 orders"

"Report sent to #data-team ✓"

Evaluating your SQL agent

Capstone requires 10 test questions. Here's how to build a proper eval set and measure correctness.

Question	Expected SQL	Expected Answer	Pass / Fail
Top 5 customers by revenue	<code>SELECT ... LIMIT 5</code>	Alice Corp, ...	✓
Orders placed yesterday	<code>SELECT COUNT(*) ...</code>	3,847	✓
Churn rate last quarter	<code>SELECT cancelled/...</code>	4.2%	✗

LangSmith

Observability + Eval

Trace every agent run. Build eval datasets from production traces. Run automated evals with human review. Official LangChain tool.

DeepEval

Unit testing for LLMs

pytest-style assertions. Metrics: correctness, faithfulness, answer relevance. CI/CD integration. SQL-specific evaluators.

Ragas

RAG + SQL eval

Automatic evaluation without labeled data. Measures context recall, answer correctness. Good for SQL agent answer quality.

Common mistakes to avoid

These are the patterns that break SQL agents in production. Fix them before going live.

Write DB user

GRANT SELECT only. Never give the agent INSERT, UPDATE, DELETE, or DROP — even for testing.

Poor tool docstrings

The LLM picks tools by reading their docstrings. Vague descriptions = wrong tool calls. Write one clear sentence per tool: what it does, when to use it.

No timeout / no LIMIT

A full-table scan with no LIMIT will block your DB. Set statement_timeout AND top_k. Both.

No eval set

“It works on my laptop” is not a release test. Build 10+ question/expected-SQL/expected-answer triplets before shipping. Run them with LangSmith or DeepEval on every change.

Raw tables, no Semantic Layer

Expose business metrics (sales_revenue, churn_rate), not raw columns. Safer and far more stable SQL.

No audit log

Log every query with user_id, generated SQL, row count, and timestamp BEFORE returning the result. Required for compliance, debugging, and LangSmith evaluation replay.

Build it: MCP + SQL agent

GOAL A LangGraph agent that answers questions by routing between an MCP-connected analytics server and a SQL agent toolkit.

Install deps

1 pip install mcp fastmcp langchain-mcp-adapters langchain langchain-community langgraph

Write the MCP server

2 Create mcp_server.py with @mcp.tool() run_query and @mcp.resource() get_schema. Run fastmcp dev to verify.

Wire the MCP client

3 MultiServerMCPClient → await client.get_tools() → create_agent(model, tools, system_prompt). Test with a question.

Build the SQL agent

4 SQLiteDatabase.from_uri() → SQLiteDatabaseToolkit(db, llm) → toolkit.get_tools() → create_agent with hub.pull prompt.

Merge tools & route

5 Pass both MCP tools and SQL toolkit tools into one create_agent call. The agent picks per question type.

Stream & probe

6 Use agent.astream(stream_mode='updates'). Ask DB questions and NL questions. Verify the agent routes correctly.

Capstone: tie it all together

THE BUILD

Internal analytics copilot

A LangGraph agent that routes between two tool sets depending on the question: MCP tools (FastMCP server via langchain-mcp-adapters) for live operational data, and SQLDatabaseToolkit tools (langchain-community) for warehouse analytics. One create_agent call, both tool sets. LangGraph is used over a one-shot agent because the capstone needs conversation memory across follow-up questions, per-session tool-call history so the agent can refine its SQL, and checkpointing so long-running analytics jobs survive interruptions.

DONE MEANS

-  FastMCP server running, connected via MultiServerMCPClient
-  SQLDatabaseToolkit wired with read-only DB user and timeout
-  Both tool sets merged into one create_agent call
-  Eval set of 10 questions routed correctly (5 MCP, 5 SQL)
-  Short README with architecture diagram

Demo day: 5-minute walkthrough + live Q&A · Code repo + working evidence required.

Where to go from here

OFFICIAL DOCS

Model Context Protocol

modelcontextprotocol.io

FastMCP

gofastmcp.com

langchain-mcp-adapters

[langchain.com → mcp-adapters](https://langchain.com/docs/mcp-adapters)

SQLDatabaseToolkit

[langchain.com → SQLDatabaseToolkit](https://langchain.com/docs/toolkits/sql)

READ NEXT

LangGraph v1.0 release notes

blog.langchain.dev/langgraph-v1-0

Anthropic: Building Effective Agents

[anthropic.com → effective-agents](https://anthropic.com/agents)

LangChain Academy

academy.langchain.com

LangGraph migration guide

[langchain.github.io → migrate-agent](https://langchain.github.io/docs/migrate-agent)

BUILD PRACTICE

Expose your team's Notion or Confluence as an MCP resource server

Add sqlglot AST validation on top of QuerySQLCheckerTool

Benchmark answer quality with Ragas on your SQL agent eval set

Open-source your FastMCP server on GitHub

Thank you.

Questions, ideas, sharp critiques — all welcome.

REACH OUT

Trainers: Nguyễn Hữu Nhật Thành · Nguyễn Ngọc Quý

Registration: training@tma.com.vn

SEE YOU ON ZOOM

11 June 2026

14:00 – 16:00 ICT